

5. Design and implement C/C++ Program to obtain the Topological ordering of vertices in a given digraph.

```
#include <stdio.h>

int k, top, t[100], i, stack[20], u, v, indegree[20];

int a[10][10], j, n;
```

```
void find_indegree();

void topological();
```

```
int main()
{
    printf("Enter the number of nodes: ");
    scanf("%d", &n);

    printf("Enter the adjacency matrix:\n");
    for(i = 1; i <= n; i++)
        for(j = 1; j <= n; j++)
            scanf("%d", &a[i][j]);

    topological();
    return 0;
}
```

```
void find_indegree()
{
    int i, j, sum;
    for(j = 1; j <= n; j++)
    {
        sum = 0;
        for(i = 1; i <= n; i++)
        {
            sum = sum + a[i][j];
        }
        indegree[j] = sum;
    }
}
```

```
void topological()
{
    k = 1;
    top = -1;

    find_indegree();

    // Push all vertices with indegree 0
    for(i = 1; i <= n; i++)
    {
        if(indegree[i] == 0)
        {
            stack[++top] = i;
        }
    }

    while(top != -1)
    {
```

```

u = stack[top--];
t[k++] = u;

for(v = 1; v <= n; v++)
{
    if(a[u][v] == 1)
    {
        indegree[v]--;
        if(indegree[v] == 0)
        {
            stack[++top] = v;
        }
    }
}

// Check for cycle
if(k <= n)
{
    printf("\nTopological ordering is not possible (Graph has a cycle)\n");
}
else
{
    printf("\nTopological ordering:\n");
    for(i = 1; i < k; i++)
    {
        printf("%d ", t[i]);
    }
    printf("\n");
}
}

```